GDG CTF

Solution Guide

Postcard from Kestrel Point (Beginner) · Chakravyuh (Advanced)

Challenge	Level	Category	Flag
Postcard from Kestrel Point	Beginner	Forensics / Stego	`GDG{l1ght_th3_w4y_thr0ugh_th3_l4y3rs}`
Chakravyuh	Advanced	Stego + OSINT + OT/ICS	`GDG{4bh1m4nyu_f0und_th3_3x1t_v14_m0dbu5}`

Every command in this guide was run against the real challenge files, and every output shown is the tool's actual output. Screenshots are from the live challenge: the Waypoint board, both Google Drive links, CyberChef, Google Maps and the final review.

Tools used (all free): exiftool, binwalk, unzip / 7-Zip, ffmpeg or Audacity, zsteg, tshark / Wireshark, Python 3, strings, CyberChef (gchq.github.io/CyberChef), Google Maps.

All systems in both challenges are fictional: a synthetic Modbus capture on private addresses, a drawn image and photo, and made-up board users. The only real places are two public heritage sites.

Part 1 · Postcard from Kestrel Point

Level: Beginner **File:** postcard.png **Flag:** `GDG{l1ght_th3_w4y_thr0ugh_th3_l4y3rs}`

A postcard from the keeper of Kestrel Point, a lighthouse that exists only in this CTF. He hid his last log entry "where only a sharp eye and a sharp ear can find it".

Chain at a glance

```
postcard.png
 +- tEXt Comment --base64--> "...password (rot13): xrrcre_bs_gur_yvtug" --rot13--> keeper_of_the_light
 +- bytes after IEND = password-protected ZIP
      +- storeroom/foghorn.wav   --spectrogram-->  "KEY = LAMP"
      +- storeroom/logbook.txt   --hex, XOR LAMP-->  GDG{l1ght_th3_w4y_thr0ugh_th3_l4y3rs}
```



Figure 1. The challenge file: looks like an ordinary postcard.

Step 1: Read the metadata

A picture is more than its pixels. exiftool shows a Base64-looking **Comment** and warns that there is data after the end of the PNG (the IEND chunk), which is the second clue.

```
$ exiftool postcard.png | grep -E 'File Size|Image Size|Title|Author|Comment|Software|Warning'
File Size                       : 786 kB
Title                           : Greetings from Kestrel Point
Author                          : The Keeper
Comment                         : VGhlIHN0b3Jlcm9vbSBpcyBsb2NrZWQuIFRoZSBwYXNzd29yZCwgaW4gdGhlIG9sZCBrZWVwZX
    IncyBjaXBoZXIgKHJvdDEzKTogeHJyY3JlX2JzX2d1cl95dnR1Zw==
Software                        : LighthouseCam 1.0
Warning                         : [minor] Trailer data after PNG IEND chunk
Image Size                      : 900x560
```

Step 2: Decode the comment (Base64, then ROT13)

```
$ echo 'VGhlIHN0b3Jlcm9vbSBpcyBsb2NrZWQuIFRoZSBwYXNzd29yZCwgaW4gdGhlIG9sZCBrZWVwZXIncyBjaXBoZXIgKHJvdDEzKTog
    eHJyY3JlX2JzX2d1cl95dnR1Zw==' | base64 -d; echo
The storeroom is locked. The password, in the old keeper's cipher (rot13): xrrcre_bs_gur_yvtug
```

The password is in "the old keeper's cipher", ROT13 (each letter shifted by 13):

```
$ echo xrrcre_bs_gur_yvtug | tr 'A-Za-z' 'N-ZA-Mn-za-m'
keeper_of_the_light
```

Password: `keeper_of_the_light`

Step 3: Find and open the hidden ZIP

binwalk shows a ZIP archive starting at offset 16729, right after the PNG ends. unzip (or 7-Zip) can open the PNG directly; it warns about the extra bytes in front and extracts anyway.

```
$ binwalk postcard.png

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             PNG image, 900 x 560, 8-bit/color RGB, non-interlaced
302           0x12E           Zlib compressed data, best compression
16729         0x4159          Zip archive data, encrypted at least v2.0 to extract, compressed size:
    768565, uncompressed size: 793844, name: storeroom/foghorn.wav
785345        0xBFBC1         Zip archive data, encrypted at least v2.0 to extract, compressed size: 313,
    uncompressed size: 459, name: storeroom/logbook.txt
785843        0xBFDB3         End of Zip archive, footer length: 22
```

```
$ unzip -P keeper_of_the_light postcard.png
Archive:  postcard.png
warning [postcard.png]:  16729 extra bytes at beginning or within zipfile
  (attempting to process anyway)
  inflating: storeroom/foghorn.wav
  inflating: storeroom/logbook.txt
```

```
$ cat storeroom/logbook.txt
KESTREL POINT LIGHT - KEEPER'S LOGBOOK (fictional)
================================================
12 Oct  Wind NW 4. Lens cleaned. Lamp lit 17:42.
13 Oct  Fog from 03:10. Horn sounded for 5 hrs.
14 Oct  Supply boat late. Paraffin low.
15 Oct  Fog again. I have put the last entry somewhere safe.
        Only someone who can SEE the foghorn will read it.

LAST ENTRY (scrambled):
0b050a2b20702a38381e39387f1e3a64351e39383e713837241e39387f1e216435723f2331
```

The last entry is hex-encoded and scrambled. The note says only someone who can **see** the foghorn can read it.

Step 4: Look at the sound (spectrogram)

Open **foghorn.wav** in Audacity (track menu → Spectrogram) or Sonic Visualiser, or render it with ffmpeg. The low foghorn tone is at the bottom; between about 3 and 9 kHz the key is written as text.

```
$ ffmpeg -loglevel error -y -i storeroom/foghorn.wav -lavfi showspectrumpic=s=1000x400 spectrogram.png &&
    echo saved spectrogram.png
saved spectrogram.png
```

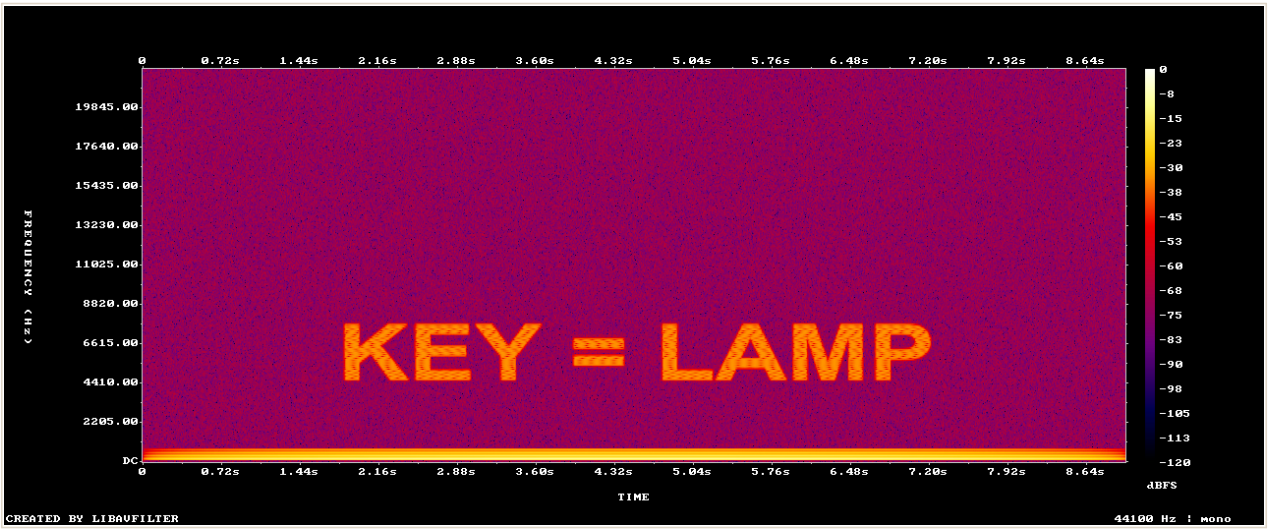


Figure 2. Spectrogram of foghorn.wav: the key KEY = LAMP is drawn between 3 and 9 kHz.

Step 5: XOR the last entry with the key

CyberChef recipe: **From Hex** → **XOR** (key LAMP, UTF8). The same in one line of Python:

```
$ python3 -c "ct=bytes.fromhex('0b050a2b20702a38381e39387f1e3a64351e39383e713837241e39387f1e216435723f2331')
    ; print(bytes(c ^ b'LAMP'[i%4] for i,c in enumerate(ct)).decode())"
GDG{l1ght_th3_w4y_thr0ugh_th3_l4y3rs}
```

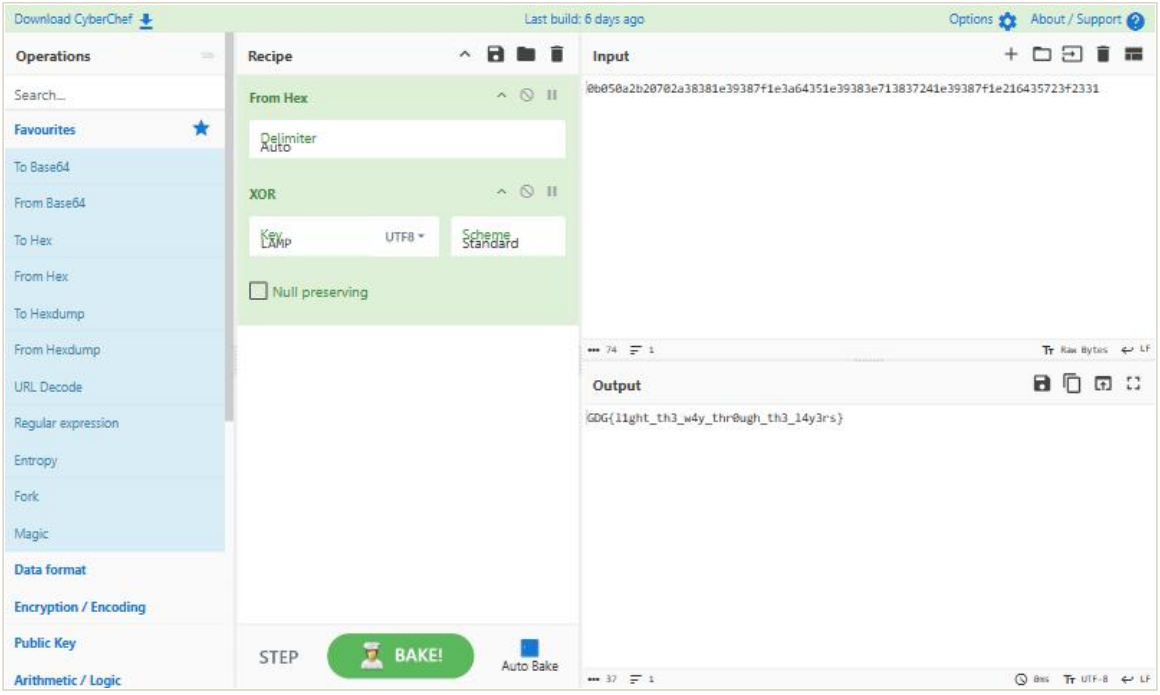


Figure 3. CyberChef: From Hex, then XOR with LAMP, gives the flag.

Flag: `GDG{l1ght_th3_w4y_thr0ugh_th3_l4y3rs}`

Part 2 · Chakravyuh

Level: Advanced **File:** chakravyuh.png **Attempts:** 10 **Flag:**
`GDG{4bh1m4nyu_f0und_th3_3x1t_v14_m0dbu5}`

Dronacharya has laid the Chakravyuh. Abhimanyu knew the way in but not the way out. A rogue machine has been whispering to a PLC in the Mayasabha Jal Kendra, a water works that exists only in this CTF. Follow the charioteer, and trust only those he tells you to trust. Shakuni has scattered loaded dice (fake flags) everywhere. The first ring opens with the name of this challenge.

Chain at a glance

```
Ring 1  chakravyuh.png --blue LSB--> base64 --XOR 'Chakravyuh'--> 29 57'41"N 76 46'16"E
Ring 2  Google Maps: Jyotisar --> Waypoint board --> parth_sarathi --Vigenere PANCHAJANYA--> Drive #1
Ring 3  ps7_capture.pcap --rogue 10.18.0.66, accepted FC6 writes to HR40--> Drive #2 link
Ring 4  mayasabha.jpg --EXIF GPS--> 28 36'34"N 77 14'39"E = Purana Qila
Ring 5  Waypoint board: Purana Qila --> vidura: 'search the Google Maps reviews for the formation'
Ring 6  Google Maps: Purana Qila reviews --search 'Chakravyuh'--> FLAG
```

The theme carries the logic: Krishna (Parthasarathi, Arjuna's charioteer) shows the way; the Maya Sabha, the palace of illusions, is the OT plant and the photo; Shakuni's loaded dice are the fake flags; and Vidura, who never lies, gives the final direction. Each of the 12 fake flags says it is fake once you read it.

Ring 1: the image

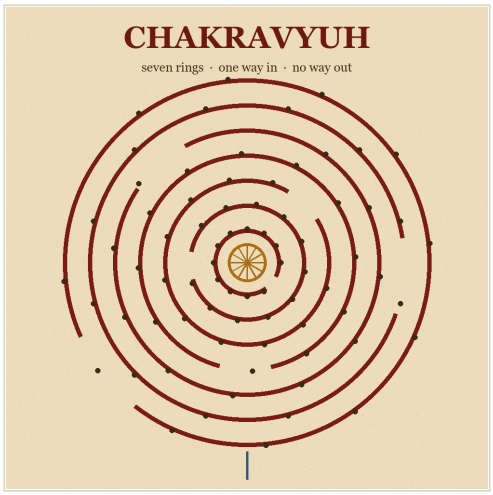


Figure 4. chakravyuh.png: the seven-ring formation.

Step 1: Metadata

```
$ exiftool chakravyuh.png | grep -E 'Image Size|Title|Author|Comment|Description'
Title                           : Chakravyuh
Author                          : Dronacharya
Comment                         : GDG{dr0n4_s4ys_l00k_d33p3r}
Description                     : Abhimanyu knew the way in. Do you know the way out?
Image Size                      : 1000x1000
```

Fake flag: `GDG{dr0n4_s4ys_l00k_d33p3r}` : read it, it admits it is a trap. Do not submit.

Step 2: Check every colour plane's lowest bit (zsteg)

zsteg finds text in two planes: the red channel and the blue channel (bit 0, row order).

```
$ zsteg chakravyuh.png 2>/dev/null | grep -E 'b1,(r|b),lsb,xy'
b1,r,lsb,xy         .. text: "R0RHe3dyMG5nX2NoNG5uM2xfcDRydGg0fQ=="
b1,b,lsb,xy         .. text: "PxRBKDogPSs0Pho9KUtIQQIREEglARMYBkEEEBsPYxQdYSYJE1kWACIaCAQGBBMLVRszBwoOUhYeHA
    cNYxwJDlIOGh1VCiIGGAocQQUNHAQvSBIfEw8SClticVGj20dWUU1ESg1IVl2w0UJPUll1SiRhPwAGClUEKg1PSyAEFx1VHCsNQTwTGA
    YWHAY3SAMEExMSWRoGYxwJAgFBNS0zSDANEx0XE1ZRWh8iEREEGw8CUFtiDAYNElIRFwsBABwb"
```

```
$ echo 'R0RHe3dyMG5nX2NoNG5uM2xfcDRydGg0fQ==' | base64 -d; echo
GDG{wr0ng_ch4nn3l_p4rth4}
```

Fake flag: `GDG{wr0ng_ch4nn3l_p4rth4}` : read it, it admits it is a trap. Do not submit.

The blue stream decodes to bytes that aren't text, so it is encrypted. The full string:

```
$ zsteg -E b1,b,lsb,xy chakravyuh.png 2>/dev/null | tr -d '\0' | grep -aoE '^[A-Za-z0-9+/=]+' | head -1
PxRBKDogPSs0Pho9KUtIQQIREEglARMYBkEEEBsPYxQdYSYJE1kWACIaCAQGBBMLVRszBwoOUhYeHAcNYxwJDlIOGh1VCiIGGAocQQUNHAQv
    SBIfEw8SClticVGj20dWUU1ESg1IVl2w0UJPUll1SiRhPwAGClUEKg1PSyAEFx1VHCsNQTwTGAYWHAY3SAMEExMSWRoGYxwJAgFBNS0z
    SDANEx0XE1ZRWh8iEREEGw8CUFtiDAYNElIRFwsBABwbABkTFR4QVRszDQAAAUECERBINxoUHxpPfA==
```

Step 3: Decrypt with the challenge name

"The first ring opens with the name of this challenge", so the key is **Chakravyuh**. CyberChef: **From Base64** → **XOR** (key Chakravyuh, UTF8), or in Python:

```
$ python3 -c "import base64; d=base64.b64decode('$BLUE'); k=b'Chakravyuh'; print(bytes(c ^ k[i%len(k)] for
    i,c in enumerate(d)).decode())"
|| CHAKRAVYUH : the first ring ||
The charioteer spoke where the old banyan still stands.
29°57'41"N 76°46'16"E
Maps lie. Read the Waypoint board on this CTF server (/waypoint).
Only parth_sarathi speaks the truth.
```

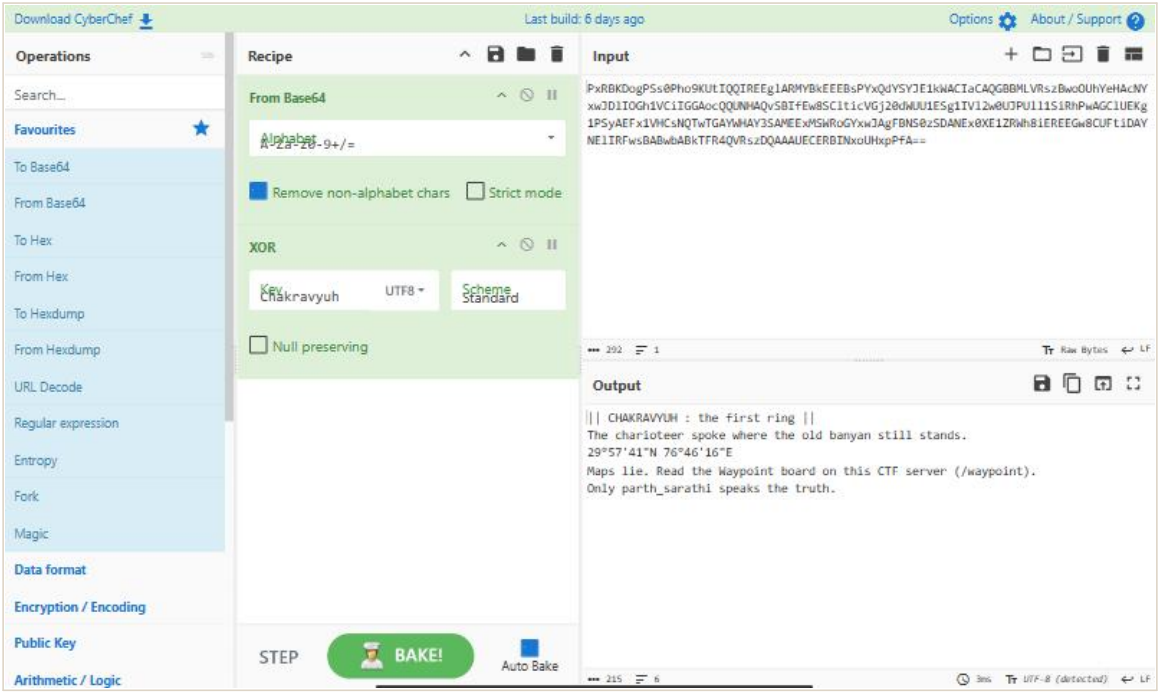


Figure 5. CyberChef: From Base64, then XOR with Chakravyuh, reveals the first coordinates.

Result: coordinates 29°57'41"N 76°46'16"E, an instruction to read the **Waypoint board**, and the one user to trust: **parth_sarathi**.

Ring 2: Jyotisar and the conch

Step 4: Reverse-geocode the coordinates

Paste the coordinates into Google Maps. The pin lands on **Jyotisar Tirth, Kurukshetra**, where the Bhagavad Gita was spoken under a banyan tree ("where the old banyan still stands").

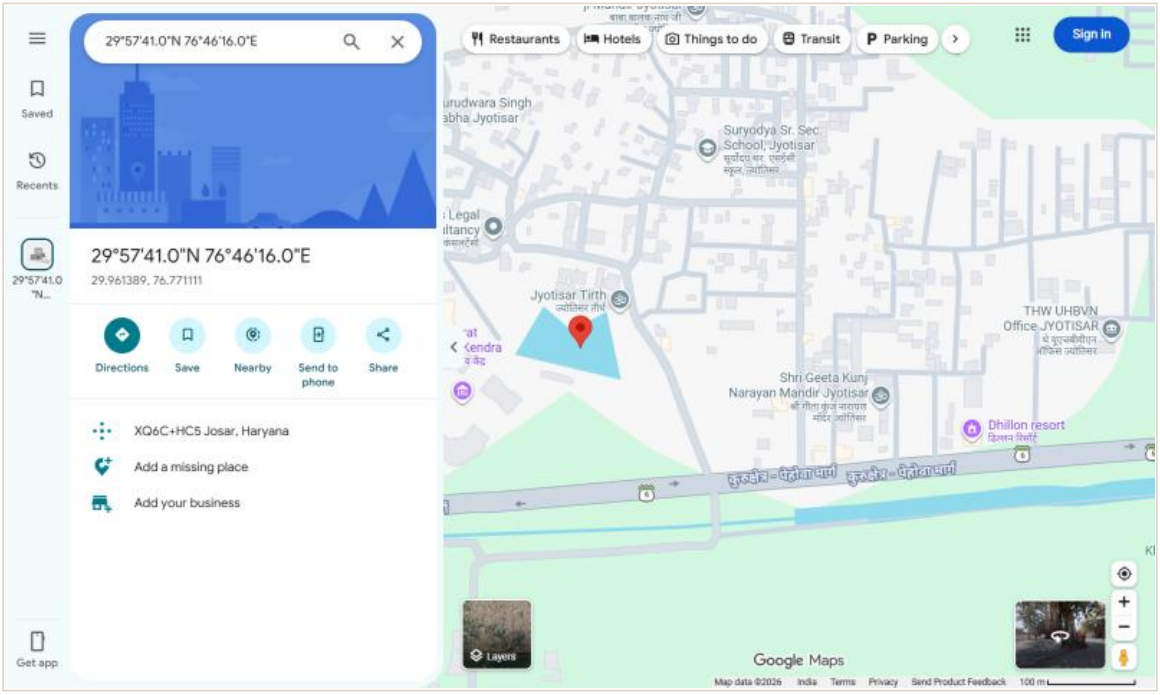


Figure 6. Google Maps: 29°57'41"N 76°46'16"E is Jyotisar Tirth.

Step 5: Read the Waypoint board

Open **/waypoint** on the CTF server and search **Jyotisar**. The board only searches by place name, never by coordinates, so the reverse-geocoding step can't be skipped.



Figure 7. Waypoint, Jyotisar pin. Only parth_sarathi's comment matters.

Fake flag: `GDG{sh4kuni_s4ys_n1c3_try}` : read it, it admits it is a trap. Do not submit.

duryodhan's base64 decodes to `GDG{duryodh4n_n3v3r_t3lls_th3_truth}`, also fake.

Step 6: Identify the cipher and find the key

In parth_sarathi's text, spaces, digits, capital letters and :// are untouched, and uvapb:// has exactly the shape of https://, but each letter is shifted by a different amount. That is a **Vigenère** cipher. "Blow my conch": Krishna's conch is **Panchajanya** (Bhagavad Gita 1.15). CyberChef: **Vigenère Decode**, key PANCHAJANYA.

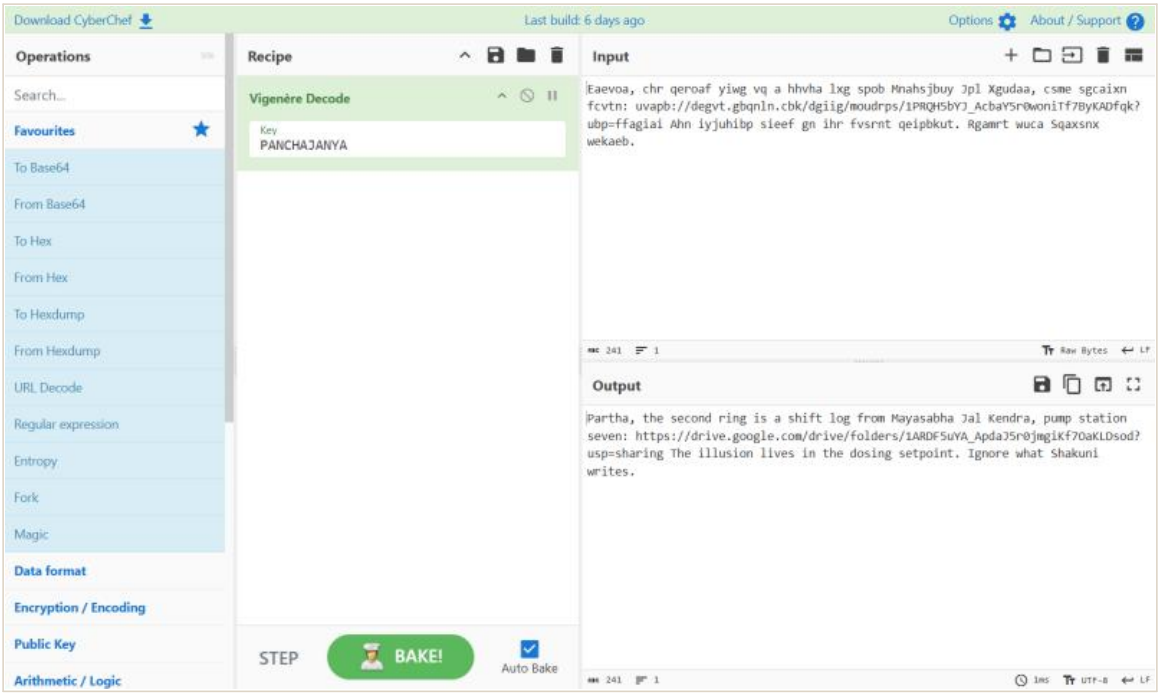


Figure 8. CyberChef Vigenère Decode with PANCHAJANYA gives the Drive #1 link and the clue 'the dosing setpoint'.

Result: Drive #1 link, plus two clues: *"The illusion lives in the dosing setpoint"* and *"Ignore what Shakuni writes"*.

Trap: the nearby Brahma Sarovar pin has `GDG{wr0ng_w4t3r_sh4kun1_w1ns}` and parth_sarathi saying "Wrong water, Partha".

Ring 3: the OT capture

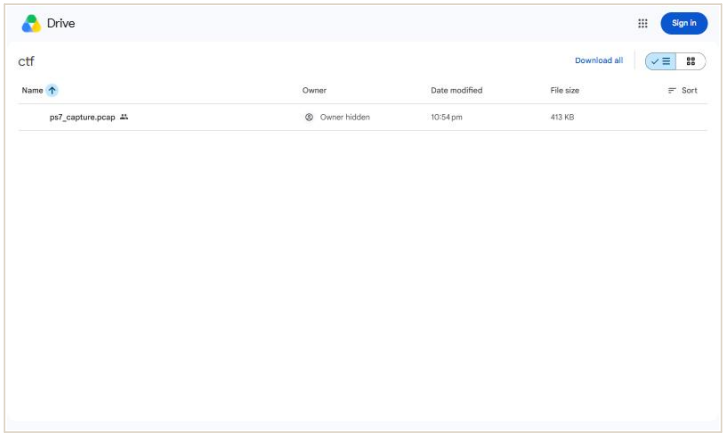


Figure 9. Drive #1: ps7_capture.pcap (413 KB).

Step 7: Find the stranger on the OT network

Statistics → Conversations → IPv4 in Wireshark (or tshark below). Four hosts poll the PLCs for the whole 7 minutes; **10.18.0.66** appears only at t = 137 s. Its TTL of 128 (Windows) stands out in a network where everything else is 64.

```
$ tshark -r ps7_capture.pcap -q -z conv,ip | sed -n '1,12p'
# (tshark's byte columns dropped to fit the page; values unchanged)
Conversation                      Frames   Start (s)  Duration (s)
10.18.0.20 <-> 10.18.0.10           3789         0.0         420.1
10.18.0.20 <-> 10.18.0.11            636         0.0         420.1
10.18.0.30 <-> 10.18.0.10            258         0.5         419.6
10.18.0.30 <-> 10.18.0.11            258         0.5         419.5
10.18.0.66 <-> 10.18.0.10            204       137.2         125.1
```

```
$ tshark -r ps7_capture.pcap -T fields -e ip.src -e ip.ttl | sort -u
10.18.0.10 64
10.18.0.11 64
10.18.0.20 64
10.18.0.30 64
10.18.0.66 128
```

Step 8: Who writes to the PLC?

Function code 6 = Write Single Register, 16 = Write Multiple Registers. The HMI (10.18.0.20) makes one legitimate write. The rogue writes to holding registers **40** and **42**, and makes one FC16 write to 3840 (0x0F00). Lines from 10.18.0.10 are the PLC's replies.

```
$ tshark -r ps7_capture.pcap -Y 'modbus.func_code==6 || modbus.func_code==16' -T fields -e ip.src -e
    modbus.func_code -e modbus.reference_num | sort | uniq -c
      1 10.18.0.10 16
      1 10.18.0.10 6
     43 10.18.0.10 6 40
     18 10.18.0.10 6 42
      1 10.18.0.20 6 40
      1 10.18.0.66 16 3840
     43 10.18.0.66 6 40
     18 10.18.0.66 6 42
```

Step 9: Find the illusions

Two writes were refused. Exception code 2 (Illegal Data Address) rejected the FC16 write. Exception code 6 (Server Device Busy) rejected one FC6 write, which the rogue then retried. There are also three TCP retransmissions of rogue requests.

```
$ tshark -r ps7_capture.pcap -Y 'modbus.exception_code' -T fields -e frame.number -e ip.src -e ip.dst -e
    mbtcp.trans_id -e modbus.func_code -e modbus.exception_code
2069 10.18.0.10 10.18.0.66 1176 16 2
2766 10.18.0.10 10.18.0.66 1203 6 6
```

```
$ tshark -r ps7_capture.pcap -Y 'tcp.analysis.retransmission' -T fields -e frame.number -e ip.src -e
    tcp.seq -e frame.len
2615 10.18.0.66 478 66
2922 10.18.0.66 634 66
3142 10.18.0.66 730 66
```

The refused FC16 write is plain ASCII, so it is what a quick **strings** search finds:

```
$ strings -n 8 ps7_capture.pcap | grep GDG
 GDG{th3_plc_r3j3ct3d_th1s_l13}
```

Fake flag: `GDG{th3_plc_r3j3ct3d_th1s_l13}` : read it, it admits it is a trap. Do not submit.

Step 10: Take only the writes the PLC accepted, on the dosing setpoint (HR40)

The cleanest filter is the **PLC's own echo replies** to the rogue. A successful FC6 reply repeats the register and value, so rejected writes (exceptions) and retransmissions drop out automatically. The last value, 125, only restores the original setpoint.

```
$ tshark -r ps7_capture.pcap -Y 'ip.src==10.18.0.10 && ip.dst==10.18.0.66 && modbus.func_code==6 &&
    modbus.reference_num==40 && !modbus.exception_code' -T fields -e modbus.regval_uint16 | tr '\n' ' ';
    echo
26740 29808 29498 12079 25714 26998 25902 26479 28519 27749 11875 28525 12134 26988 25903 25647 12609 21076
    24920 14714 19276 27762 16685 17250 19546 20289 14407 22627 25157 30566 22630 30518 27183 30313 25975
    16245 29552 15731 26721 29289 28263 125
```

Each 16-bit register holds two ASCII characters (big-endian). Drop the final 125 and convert:

```
$ tshark -r ps7_capture.pcap -Y 'ip.src==10.18.0.10 && ip.dst==10.18.0.66 && modbus.func_code==6 &&
    modbus.reference_num==40 && !modbus.exception_code' -T fields -e modbus.regval_uint16 | head -n -1 |
    python3 -c "import sys; print(b''.join(int(v).to_bytes(2,'big') for v in
    sys.stdin.read().split()).decode())"
https://drive.google.com/file/d/1ARTaX9zKLlrA-CbLZOA8GXcbEwfXfw6j/view?usp=sharing
```

Result: the Drive #2 link: `https://drive.google.com/file/d/1ARTaX9zKLlrA-CbLZOA8GXcbEwfXfw6j/view`

What goes wrong if you take the rogue's *requests* instead of the PLC's accepted replies: the busy-rejected write is counted twice, and the link breaks (`...lrA-CbCbLZOA8...`):

```
$ tshark -r ps7_capture.pcap -Y 'ip.src==10.18.0.66 && modbus.func_code==6 && modbus.reference_num==40' -T
    fields -e modbus.regval_uint16 | head -n -1 | python3 -c "import sys;
    print(b''.join(int(v).to_bytes(2,'big') for v in sys.stdin.read().split()).decode())"
https://drive.google.com/file/d/1ARTaX9zKLlrA-CbCbLZOA8GXcbEwfXfw6j/view?usp=sharing
```

The other register, HR42, is the honeypot:

```
$ tshark -r ps7_capture.pcap -Y 'ip.src==10.18.0.10 && ip.dst==10.18.0.66 && modbus.func_code==6 &&
    modbus.reference_num==42 && !modbus.exception_code' -T fields -e modbus.regval_uint16 | head -n -1 |
    python3 -c "import sys; print(b''.join(int(v).to_bytes(2,'big') for v in
    sys.stdin.read().split()).decode(errors='replace'))"
GDG{m4y4_s4bh4_1llus10n_n0t_r34l}
```

Fake flag: `GDG{m4y4_s4bh4_1llus10n_n0t_r34l}` : read it, it admits it is a trap. Do not submit.

Ring 4: the photo

Step 11: Download the photo from Drive #2

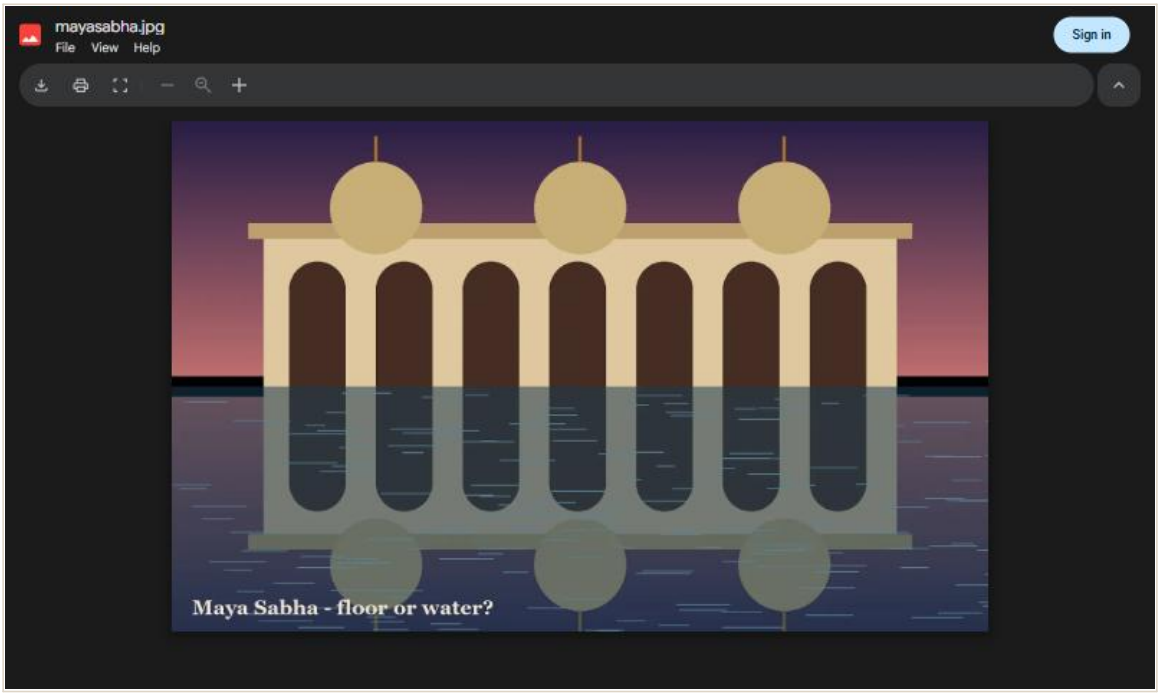


Figure 10. Drive #2: mayasabha.jpg, the Maya Sabha, 'floor or water?'. Download the original file; a screenshot loses the metadata.

Step 12: Read its EXIF metadata

```
$ exiftool mayasabha.jpg | grep -E 'Image Size|Make|Camera Model|Artist|Description|XP Comment|GPS Latitude
    |GPS Longitude  |GPS Position'
Image Description               : What the visitors say about this place is true. - Vidura
Make                            : Mayasabha Jal Kendra
Camera Model Name               : PS7 shift camera
Artist                          : Maya Danava
XP Comment                      : GDG{y0u_f3ll_1n_th3_m4y4_p00l}
Image Size                      : 1600x1000
GPS Latitude                    : 28 deg 36' 34.00" N
GPS Longitude                   : 77 deg 14' 39.00" E
GPS Position                    : 28 deg 36' 34.00" N, 77 deg 14' 39.00" E
```

Fake flag: `GDG{y0u_f3ll_1n_th3_m4y4_p00l}` : read it, it admits it is a trap. Do not submit.

The description points to the visitors' words. The GPS tag gives the second location.

Step 13: Reverse-geocode the GPS position

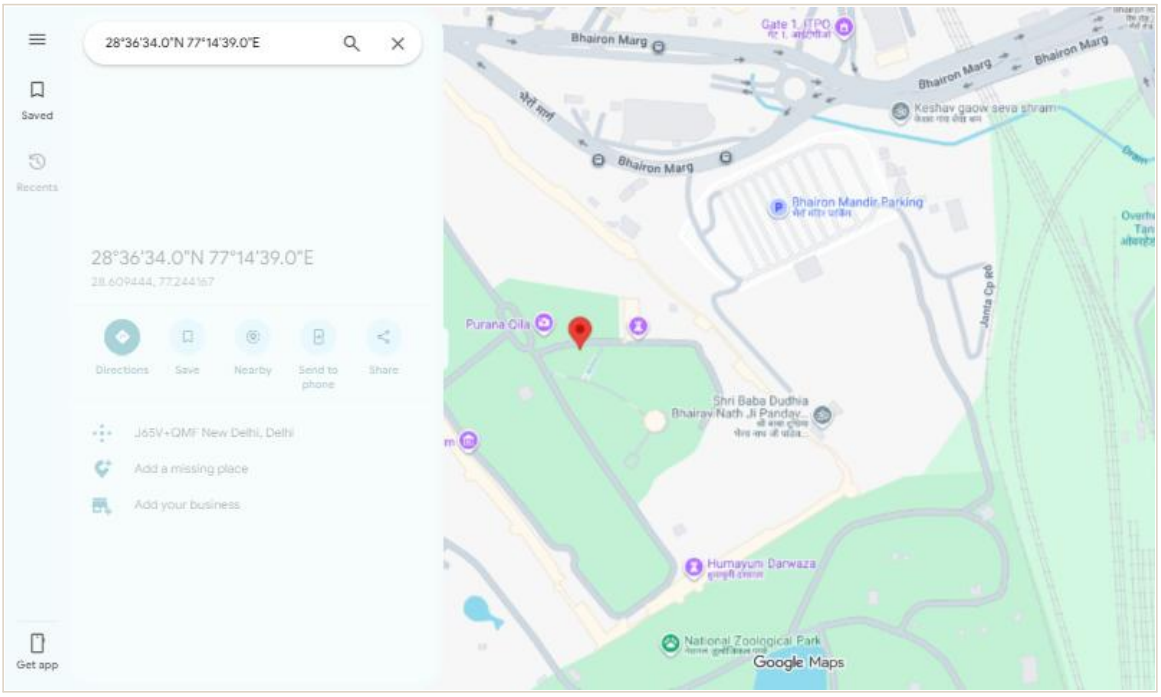


Figure 11. Google Maps: 28°36'34"N 77°14'39"E is Purana Qila, New Delhi, the traditional site of Indraprastha.

Ring 5: the last board pin

Step 14: Search Purana Qila on the Waypoint board

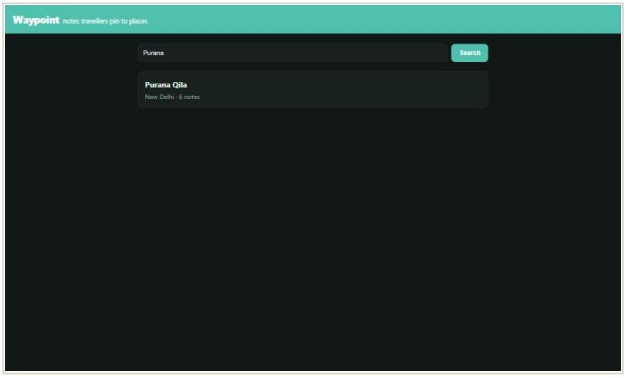


Figure 12. The board searches by place name only.

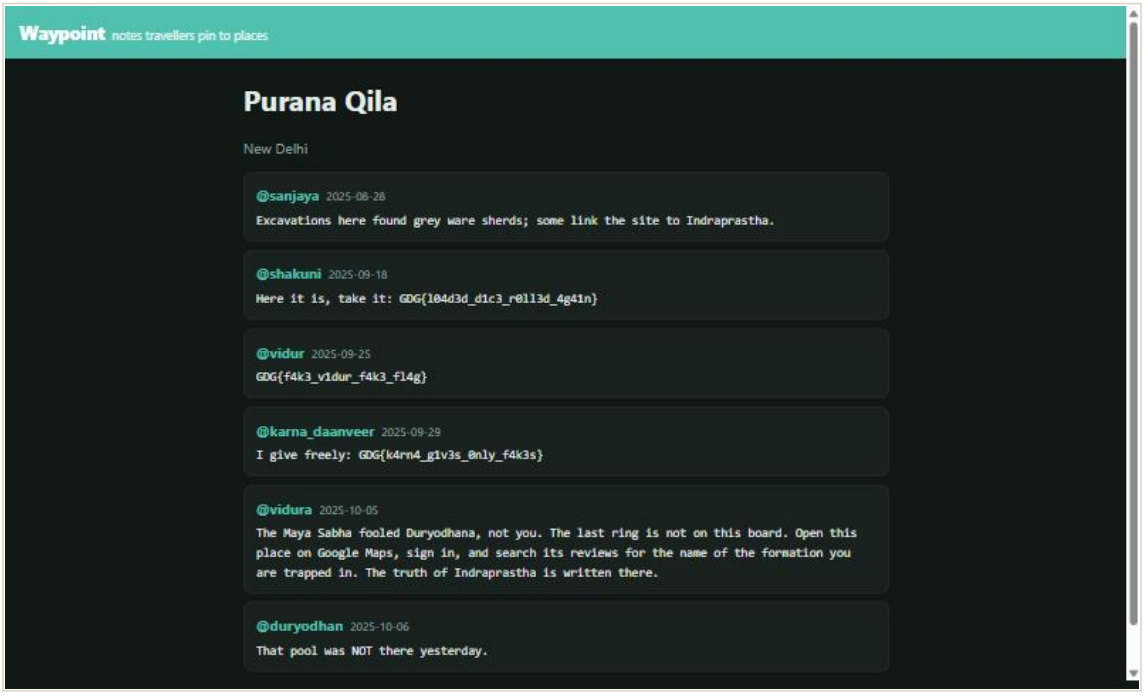


Figure 13. Purana Qila pin: three fakes, plus vidura's direction to the Google Maps reviews.

shakuni `GDG{104d3d_d1c3_r0ll3d_4g41n}`, karna_daanveer `GDG{k4rn4_g1v3s_0nly_f4k3s}` and the look-alike **vidur** `GDG{f4k3_v1dur_f4k3_fl4g}` are all fake. The real **vidura** says: *"The last ring is not on this board. Open this place on Google Maps, sign in, and search its reviews for the name of the formation you are trapped in."*

Ring 6: the Google Maps review

Step 15: Search the reviews for the formation's name

Open **Purana Qila** on Google Maps and sign in to Google (signed out, Maps shows only a few reviews and blocks search). Go to **Reviews** → **Search reviews** and type **Chakravyuh**.

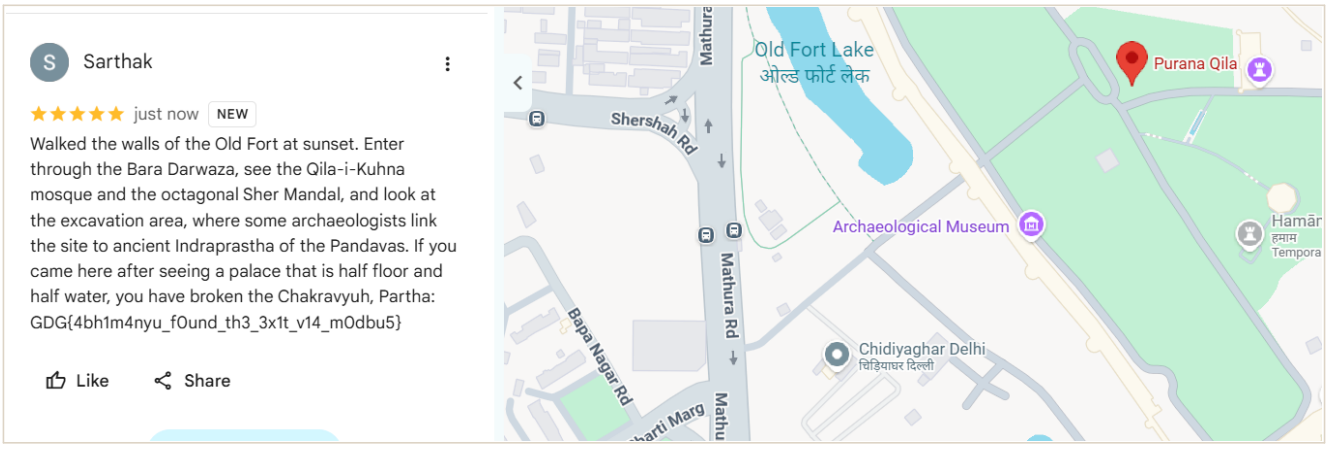


Figure 14. The review on Purana Qila holding the real flag.

Step 16: Submit

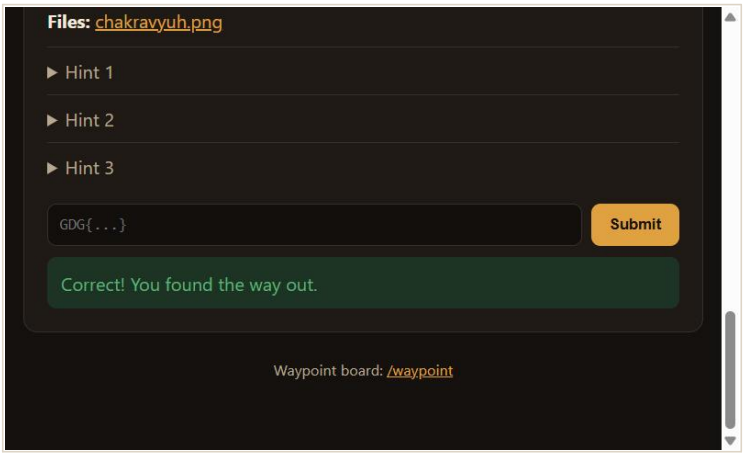


Figure 15. The CTF portal accepts the flag.

Flag: `GDG{4bh1m4nyu_f0und_th3_3x1t_v14_m0dbu5}`

Appendix: all fake flags in Chakravyuh

The portal's flag checker recognises each of these and replies "That is one of Shakuni's loaded dice" (it still costs an attempt).

#	Fake flag	Where
1	`GDG{dr0n4_s4ys_l00k_d33p3r}`	PNG tEXt Comment
2	`GDG{wr0ng_ch4nn3l_p4rth4}`	Red-channel LSB
3	`GDG{sh4kuni_s4ys_n1c3_try}`	Waypoint, Jyotisar (shakuni)
4	`GDG{duryodh4n_n3v3r_t3lls_th3_truth}`	Waypoint, Jyotisar (duryodhan, base64)
5	`GDG{wr0ng_w4t3r_sh4kun1_w1ns}`	Waypoint, Brahma Sarovar (shakuni)
6	`GDG{th3_plc_r3j3ct3d_th1s_l13}`	pcap: rejected FC16 write (strings bait)
7	`GDG{m4y4_s4bh4_1llus10n_n0t_r34l}`	pcap: honeypot holding register 42
8	`GDG{y0u_f3ll_1n_th3_m4y4_p00l}`	Photo EXIF XP Comment
9	`GDG{l04d3d_d1c3_r0ll3d_4g41n}`	Waypoint, Purana Qila (shakuni)
10	`GDG{k4rn4_g1v3s_0nly_f4k3s}`	Waypoint, Purana Qila (karna_daanveer)
11	`GDG{f4k3_v1dur_f4k3_fl4g}`	Waypoint, Purana Qila (vidur, look-alike user)
12	`GDG{t00_f4r_fr0m_1ndr4pr4sth4}`	Waypoint, National Zoological Park (shakuni)

Key values

What	Value
Ring 1 XOR key	`Chakravyuh` (the challenge name)
First coordinates	29°57'41"N 76°46'16"E = Jyotisar, Kurukshetra
Vigenère key	`PANCHAJANYA` (Krishna's conch)
Rogue host	10.18.0.66 (TTL 128)
Data register	Holding register 40 (dosing setpoint), FC6, accepted writes only
Traps in the pcap	3 TCP retransmissions, 1 busy exception (code 6), 1 rejected FC16 (code 2), HR42 honeypot, final restore write 125
Second coordinates	28°36'34"N 77°14'39"E = Purana Qila, New Delhi (photo EXIF GPS)
Final search	Google Maps, Purana Qila, Reviews, search `Chakravyuh`